GIT LOCAL

Guía práctica para crear y administrar repositorios sin GitHub

Pensada para trabajar con **Git Bash en Windows**, tanto con archivos de texto como con documentos PDF, DWG y otros archivos binarios. Los ejemplos utilizan la rama **master**.

Git funciona completamente en tu computadora: no requiere cuenta, conexión a Internet, servidor remoto ni comandos **push** o **pull**.

Recorrido: crear el repositorio → agregar archivos → registrar commits → consultar el historial → recuperar versiones → administrar exclusiones y ramas.

1. Qué guarda Git y dónde

Git mantiene un historial de **commits**: instantáneas de los archivos registrados. La carpeta oculta **.git/** contiene el repositorio y su historial; no la borres si querés conservar las versiones.

Tres espacios importantes: **carpeta de trabajo** (archivos que abrís en Windows), **staging** (cambios preparados con git add) y **último commit, HEAD** (última versión confirmada en la rama actual).

2. Crear un repositorio nuevo

Abrir Git Bash en la carpeta correcta

Entrá en la carpeta del proyecto desde el Explorador de Windows y abrí Git Bash allí; o desplazate usando **cd**. Las rutas de Git Bash usan barras **/**.

```
pwd
ls -la
git init
```

git init crea el repositorio local. La rama inicial puede llamarse master, main u otro nombre, según la configuración. No es necesario renombrarla para que Git funcione.

```
git status
git branch --show-current
```

Si querés crear un repositorio nuevo con master de forma explícita, podés usar **git init -b master**. Si ya hiciste git init, no necesitás repetirlo.

Configuración de autor (una sola vez)

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu-correo@ejemplo.com"
```

Estos datos identifican al autor de los commits; no requieren GitHub ni envían nada a Internet. Para ver la configuración: **git config --list**.

3. Registrar la primera versión

```
git status
git add Guia_Git_GitHub.pdf
git commit -m "Primera version de la guia"
```

git add prepara el archivo; **git commit** guarda esa versión en el historial local. El primer commit aparece marcado como **root-commit**: significa que no tiene un commit anterior.

Si querés agregar todos los cambios de la carpeta actual y sus subcarpetas, usá **git add .**. Revisá antes con **git status** para evitar agregar archivos innecesarios.

Qué significa -m

```
git commit -m "Actualizar guia"
git commit --message "Actualizar guia"
```

Ambos comandos son equivalentes: **-m** es la opción corta de **--message**. Sin -m, **git commit** abre un editor para escribir el mensaje. En cambio, **git commit "texto"** interpreta "texto" como una ruta, no como el mensaje, y puede fallar.

4. Rutina de trabajo diaria

```
git status
git diff
git add .
git diff --staged
git commit -m "Describir los cambios"
```

git diff muestra cambios no preparados, principalmente en archivos de texto. **git diff --staged** muestra lo preparado para el commit. En archivos binarios, como PDF o DWG, Git normalmente no muestra un diff textual útil, pero sí guarda sus versiones.

Cómo leer git status

working tree clean: no hay cambios pendientes en archivos seguidos ni nuevos archivos sin ignorar. **Changes not staged**: cambios todavía sin preparar. **Changes to be committed**: cambios preparados. **Untracked files**: archivos nuevos todavía no seguidos.

5. Consultar el historial

```
git log --oneline
git log --oneline --graph --decorate --all
git show --stat 0843fcc
git show --name-only 0843fcc
```

Cada commit tiene un identificador o **hash**; normalmente alcanza con sus primeros caracteres si son únicos. **HEAD** apunta al commit actual de la rama activa. **git show --stat** informa qué archivos cambiaron en un commit.

Marcar versiones con tags

```
git tag V1.0
git tag V1.1 0843fcc
git tag --list
```

Un **tag** es un nombre fijo para identificar un commit, como V1.0. No es lo mismo que el mensaje del commit. Es especialmente útil para localizar versiones validadas.

6. Consultar una versión anterior sin cambiar de rama

Recuperar temporalmente un archivo

```
git status
git restore --source=0843fcc --worktree -- Guia_Git_GitHub.pdf
```

El PDF de tu carpeta de trabajo pasa a tener el contenido del commit indicado. **HEAD y la rama master no se mueven**. Si lo abrís solo para consultarlo, no necesitás crear un commit.

Recuperar temporalmente todo el repositorio

```
git restore --source=75d6ebf --worktree -- :/
```

El símbolo **:/** representa las rutas desde la raíz del repositorio; **.** se refiere a la carpeta actual y su contenido. La opción **--worktree** modifica los archivos de la carpeta de trabajo, no el staging.

Volver a la última versión

```
git restore --source=HEAD --worktree -- :/
```

Esto devuelve los archivos registrados al contenido del último commit de la rama actual. Si no preparaste cambios con git add, también puede abreviarse como **git restore -- :/**. La forma explícita con **--source=HEAD** evita confusiones con el staging.

Atención: Estas operaciones pueden sobrescribir cambios locales no guardados en un commit. Antes, ejecutá git status. No eliminan automáticamente archivos sin seguimiento.

7. Recuperar una versión anterior como versión vigente

Si querés que la próxima versión del proyecto tenga el mismo contenido que una versión antigua **sin borrar el historial**, restaurá el contenido y creá un nuevo commit.

```
git status
git restore --source=0843fcc --staged --worktree -- :/
git commit -m "Restaurar version anterior"
```

Por ejemplo: versión 1 = A, versión 2 = B, versión 3 = C, versión 4 = A. La versión 4 contiene lo mismo que la 1, pero es un **commit nuevo**; las versiones 2 y 3 siguen existiendo en el historial.

Si solo querés restaurar el PDF, reemplazá **:/** por **Guia_Git_GitHub.pdf**. **--staged** prepara los cambios y **--worktree** actualiza los archivos de Windows.

Alternativa de mayor riesgo: git reset --hard

```
git reset --hard 0843fcc
```

Atención: Este comando mueve la rama actual al commit indicado y descarta cambios locales del índice y los archivos seguidos. Los commits posteriores dejan de estar en el historial visible de esa rama. No lo uses como método habitual para recuperar versiones; preferí restore + commit.

8. Comprender las opciones de los comandos

-m es una opción corta; **--message** es su forma larga. **--source=HASH** indica de qué commit tomar los archivos. **--staged** actúa sobre el área de preparación. **--worktree** actúa sobre la carpeta de trabajo.

```
git restore --source=0843fcc -- Guia_Git_GitHub.pdf
```

El **--** escrito solo separa las opciones de las rutas. En el ejemplo, **--source=0843fcc** es una opción; **--** indica que lo siguiente es una ruta. Sin especificar --staged ni --worktree, restore actúa por defecto sobre la carpeta de trabajo.

En **git restore archivo**, la fuente predeterminada es el staging. Si no hay diferencias entre staging y HEAD, el resultado coincide con el último commit. En **git restore --staged archivo**, la fuente predeterminada es HEAD y se quita el cambio del staging sin descartar el archivo modificado en Windows.

9. Ignorar archivos y carpetas con .gitignore

Creá un archivo llamado **.gitignore** en la raíz del repositorio. Ejemplo:

```
# Carpetas en cualquier nivel
pruebas/
copias_seguridad/
```

```
# Carpeta solo en la raiz
/temporales/

# Archivos por extension
*.bak
*.tmp
```

La barra final **/** indica directorio. La barra inicial **/** ancla la regla al lugar donde está el .gitignore. Las reglas pueden escribirse aunque las carpetas todavía no existan.

Si los archivos ya estaban seguidos

```
git rm -r --cached pruebas/
git add .gitignore
git commit -m "Ignorar carpeta de pruebas"
```

--cached quita los archivos del seguimiento actual de Git sin borrarlos del disco. Sus versiones anteriores permanecen en el historial. Un .gitignore por sí solo no deja de seguir archivos ya registrados.

```
git check-ignore -v pruebas/archivo.txt
```

Este comando permite identificar qué regla ignora un archivo. Si el repositorio solo contiene un PDF y no genera archivos auxiliares, el .gitignore es opcional.

10. Ramas locales y worktrees

Las ramas permiten desarrollar alternativas sin modificar directamente master. Un **worktree** permite tener otra versión o rama en una carpeta separada, compartiendo el historial del mismo repositorio.

```
git branch
git switch -c prueba
git switch master
```

Abrir una versión antigua en otra carpeta

```
git worktree add --detach ../PROYECTO_ANTERIOR V1.0
git worktree list
```

Con **--detach** consultás el commit o tag sin crear una rama nueva. Si pasás un tag directamente, Git también suele usar detached HEAD. Esto no cambia la versión vigente de master.

Crear una rama alternativa desde una versión antigua

```
git worktree add -b prueba-v1 ../PROYECTO_PRUEBA V1.0
```

Esto crea la rama local **prueba-v1** y la abre en otra carpeta; master no cambia. Para retirar un worktree registrado y limpio: **git worktree remove ../PROYECTO_PRUEBA**. Revisá primero con **git worktree list**.

Atención: Si la carpeta está sincronizada con OneDrive, los archivos bloqueados pueden impedir eliminar un worktree. No borres la carpeta principal del repositorio ni su .git.

11. Tabla de consulta rápida

Objetivo	Comando
Crear repositorio	git init
Ver estado	git status
Preparar archivo	git add archivo
Guardar versión	git commit -m "mensaje"
Ver historial	git log --oneline
Consultar versión vieja	git restore --source=HASH --worktree -- :/
Volver a HEAD	git restore --source=HEAD --worktree -- :/
Recuperar y confirmar	git restore --source=HASH --staged --worktree -- :/
Ver tags	git tag --list
Ver ramas	git branch
Ver worktrees	git worktree list

Consejo: para aprender con seguridad, hacé pruebas en un repositorio descartable y consultá **git status** antes de ejecutar comandos que reemplacen archivos.